Polymath15, fifth thread: finishing off the test problem?

Terence Tao · 2 March 2018

https://terrytao.wordpress.com/2018/03/02/polymath15-fifth-thread-finishing-off-the-test-problem/

tao2typ · converted to Typst 2026-10-14 (UTC)

This is the fifth "research" thread of the Polymath15 project to upper bound the de Bruijn-Newman constant Λ, continuing this post. Discussion of the project of a non-research nature can continue for now in the existing proposal thread. Progress will be summarised at this Polymath wiki page.

We have almost finished off the test problem of showing that $H_t(x+iy) \neq 0$ whenever $t = y = 0.4$. We have two useful approximations for H_t, which we have denoted $A^{eff} + B^{eff}$ and $A^{eff} + B^{eff} - C^{eff}$, and a normalising quantity B_0^{eff} that is asymptotically equal to the above expressions; see the wiki page for definitions. In practice, the $A^{eff} + B^{eff}$ approximation seems to be accurate within about one or two significant figures, whilst the $A^{eff} + B^{eff} - C^{eff}$ approximation is accurate to about three or four. We have an effective upper bound

$$\left|H_t - A^{eff} - B^{eff}\right| \leq E_1 + E_2 + E_3^*$$

where the expressions E_1, E_2, E_3^* are quite small in practice (E_3^* is typically about two orders of magnitude smaller than the main term B_0^{eff} once x is moderately large, and the error terms E_1, E_2 are even smaller). See this page for details. In principle we could also obtain an effective upper bound for $\left|H_t - (A^{eff} + B^{eff} - C^{eff})\right|$ (the E_3^* term would be replaced by something smaller).

The ratio $\frac{A^{eff}+B^{eff}}{B_0^{eff}}$ takes the form of a difference $\sum_{n=1}^N \frac{b_n}{n^s} - e^{i\theta} \sum_{n=1}^N \frac{a_n}{n^s}$ of two Dirichlet series, where $e^{i\theta}$ is a phase whose value is explicit but perhaps not terribly important, and the coefficients b_n, a_n are explicit and relatively simple (b_n is $\exp(\frac{t}{4}\log^2 n)$, and a_n is approximately $(n/N)^y b_n$). To bound this away from zero, we have found it advantageous to mollify this difference by multiplying by an Euler product $\prod_{p \leq P}(1 - \frac{b_p}{p^s})$ to cancel much of the initial oscillation; also one can take advantage of the fact that the b_n are real and the a_n are (approximately) real. See this page for details. The upshot is that we seem to be getting good lower bounds for the size of this difference of Dirichlet series starting from about $x \geq 5 \times 10^5$ or so. The error terms E_1, E_2, E_3^* are already quite small by this stage, so we should soon be able to rigorously keep H_t from vanishing at this point. We also have a scheme for lower bounding the difference of Dirichlet series below this range, though it is not clear at present how far we can continue this before the error terms E_1, E_2, E_3^* become unmanageable. For very small x we may have to explore some faster ways to compute the expression H_t, which is still difficult to compute directly with high accuracy. One will also need to bound the somewhat unwieldy expressions E_1, E_2 by something more manageable. For instance, right now these quantities depend on the continuous variable x; it would be preferable to have a quantity that depends only on the parameter $N = \left\lfloor \sqrt{\frac{x}{4\pi} + \frac{t}{16}} \right\rfloor$, as this could be computed numerically for all x in the remaining range of interest quite quickly.

As before, any other mathematical discussion related to the project is also welcome here, for instance any summaries of previous discussion that was not covered in this post.

Comments (121)

 **Terence Tao** · 2 March 2018

As a preliminary step towards bounding E_1, E_2 by more tractable quantities, I have computed in http://michaelnielsen.org/polymath1/index.php?title=Controlling_H_t-A-B/B_0#Estimation_of_E_1.2CE_2 an upper bound on $E_1/\left|B_0^{eff}\right|$ and $E_2/\left|B_0^{eff}\right|$ that depends only on N (and on t, y of course) and not directly on x or T, thus the bound is uniform in each interval $I_N := [(4\pi N^2) - \frac{\pi t}{4}, (4\pi(N+1)^2) - \frac{\pi t}{4})$. The formula is a bit messy, but looks like it can be computed numerically for N up to say 1000, and then we can use some other means of estimating E_1, E_2 for larger values of N (at that point these errors are very small, and our lower bound for $\left|A^{eff} + B^{eff}\right|/\left|B_0^{eff}\right|$ fairly good, so probably even a very crude bound would suffice.

 **KM** · 2 March 2018

To establish a conservative upper bound for (|E1|+|E2|+|E3|)/|B0_eff| depending only on N, should we use two T's, T1 and T2? Where T1=(x_N)/2 and T2 = (x_(N+1))/2 (and corresponding T'1 and T'2). I was thinking where T or T' is present in the denominator (as in E1 or E2 and most of E3 calculations), T1 and T'1 should be used, and where T' is present in the numerator (as in E3 T'^(3/2)calculation), T'2 should be used.
Also, it seems that |E3| is not present in normalized form like |E1|/|B0_eff| and |E2|/|B0_eff|. Is there a bound on B0_eff depending only on N that can be used?

 **Terence Tao** · 3 March 2018

Yes, this is a good general strategy (using upper and lower bounds on T), though for E_1 and E_2 it is a little tricky due to the presence of the complex quantity $\alpha_1(\frac{1\pm y+ix}{2})$ in the expression.
As for $E_3^*/\left|B_0^{eff}\right|$, it is probably a good idea to factor this as $E_3^*/((T')^{3/2}e^{-\pi T/4})$ and $((T')^{3/2}e^{-\pi T/4})/\left|B_0^{eff}\right|$. The former should be comparable to 1 and the latter should be decreasing in x once x is moderately large (this should be provable rigorously, by computing the log-derivative of this latter ratio; I'll try to do this when I have a bit more time this weekend, but I think it should also be visible numerically, as this ratio is fairly easy to calculate). So one can bound this latter ratio by replacing x by the minimum value $x_N = 4\pi N^2 - \frac{\pi t}{4}$. For the former ratio one replaces T similarly with $T_N/2$ (and a with $\sqrt{T_N/2\pi}$).

 **KM** · 3 March 2018

I have been able to get a good match for the normalized E1, E2, and E3 data in the wiki (powers of 10). Was able to generate for all N from 3 to 2000, |E1|/|B0_eff| and |E2|/|B0_eff| values, and these are kept as a table in the python research folder in the repo. Even for N as low as 16, both values go below 10^-3.
Also, while B0_eff seems to monotonically decrease beyond x as low as 5 (verified this for x at intervals of 0.1 upto 100k), I realized after some basic exercises that E3 at T1 and B0_eff at T2 can't be directly divided as the B0_eff decay is exponential and the ratio gets astronomically large. So I started evaluating |E3|/|B0_eff| for all x at intervals of 0.1 upto x=1 million, which is not yet complete but going pretty fast. Will now try out the approach you outlined above.

 **Terence Tao** · 3 March 2018

Thanks for this data. Your data for |E^*_3| / |B^0_{eff}| can presumably be checked against Rudolph's graphs at https://terrytao.wordpress.com/2018/02/24/polymath15-

fourth-thread-closing-in-on-the-test-problem/#comment-493384, which also indicate that this ratio should be monotone decreasing in time.



KM · 3 March 2018

Match with Rudolph's |E3/B0| graphs (red curves) looks quite good.
For dependence only on N, taking T as x_N/2 and breaking |E3/B0| into two factors, we see that the first one is indeed comparable to 1. It decreases and settles around 0.21, beyond which it decreases quite slowly.
The second factor is the main decay term in the ratio and decreases faster (not as fast as the normalized E1 or E2 error bounds). Thus, as expected the overall normalized error bound |E/B0| follows the lead of the |E3/B0| bound.
The table with normalized E1, E2, E3 factors, E3 and overall error bounds is kept alongside the previous one.
Looking at |E/B0| (last column), we see that it is
below 0.1 at N=16,
below 0.01 at N=67,
below 0.001 at N=253,
below 0.0001 at N=830



Terence Tao · 4 March 2018

Great! Combined with the lower bounds we are able to establish on $\left|A^{eff} + B^{eff}\right|/\left|B_0^{eff}\right|$, this should rigorously establish non-vanishing of H_t in a range something like $300 \leq N \leq 2000$. To handle the large range $N \geq 2000$ we need rigorous analytic upper bounds on $E_1/\left|B_0^{eff}\right|$, $E_2/\left|B_0^{eff}\right|$, $E_3^*/\left|B_0^{eff}\right|$ and rigorous analytic lower bounds on $\left|A^{eff} + B^{eff}\right|/\left|B_0^{eff}\right|$ but there seems to be a fair amount of room available (particularly with regards to the upper bounds on error terms). I can work on these.
For intermediate values of N, e.g. $20 \leq N \leq 300$, the error bounds seem good enough that the adaptive mesh strategy to numerically lower bound $\left|A^{eff} + B^{eff}\right|/\left|B_0^{eff}\right|$ should be sufficient. This leaves the very small values of N, something like $N \leq 20$. One can either proceed here by using the $A^{eff} + B^{eff} - C^{eff}$ approximation, in which case we have to obtain explicit error terms on the accuracy of this approximation, or we can try to compute H_t directly on a mesh. The latter strategy needs a rigorous upper bound on the first derivative of H_t. One way to do this which is straightforward but a bit tedious is to repeat the error analysis on $H_t - A^{eff} - B^{eff}$ to also give upper bounds on $H_t' - (A^{eff})' - (B^{eff})'$. The Cauchy integral formula already gives some bound this way, but this is probably too lousy a bound (as we would have to vary y to use this formula). But I think a direct repetition of the arguments will give a reasonable bound, though I'm not really looking forward to the computation.



Rudolph · 4 March 2018

I also got the E1 and E2 code working in Pari/gp and do get an exact match with KM's csv-data.
Here is a plot for $t = y = 0.4$, that shows both normalised E1, E2-error terms and their bounds for $500 \leq x \leq 5000$. For each $x + 0.4i$, I established its associated N and used that to compute the bound-value (hope that is correct). The bound therefore shows as a step function and I guess there might 'smoother' ways to visualize this.
The bounds do seem to work pretty well.
https://ibb.co/kkpODS

KM · 4 March 2018

Thanks Prof. Tao. For intermediate N values with the mesh, do we also need a separate calculation of the derivative at x or an upper bound for N (like calculated for (A_toy+B_toy)/B_toy) or are the bounds obtained then reusable? I used a symbolic derivative procedure to calculate d/dx(B_eff/B0_eff) and got an automated expression (complex valued, but modulus can also be obtained)
exp((t/4)*log(n)**2) * (n**(-t*(0.5*log((0.5*I*x + y/2 + 1/2)/(2*pi)) + 1/(I*x + y + 1) + 1/(0.5*I*x + y/2 − 1/2))/2 − 0.5*I*x − y/2 − 1/2)*(-t*(-I/(I*x + y + 1)**2 + 0.25*I/(0.5*I*x + y/2 + 1/2) − 0.5*I/(0.5*I*x + y/2 − 1/2)**2)/2 − 0.5*I)*log(n))
The values at different x match well with those of d/dx(B_toy/B0_toy), and the agreement gets better as x increases.
Also, the automated expression for d/dx(A_eff/B0_eff) is quite long and not posting it here. I haven't been able to match it's values yet with those of d/dx(A_toy/B0_toy).

Terence Tao · 4 March 2018

Hmm. The ratio B^{eff}/B_0^{eff} takes the form
$\sum_{n=1}^N \frac{1}{n^{(1+y-ix)/2+\frac{t}{2}\alpha_1(\frac{1+y-ix}{2})-\frac{t}{4}\log n}}$
compared against the ratio B^{toy}/B_0^{toy}, which takes the form
$\sum_{n=1}^N \frac{1}{n^{(1+y-ix)/2+\frac{t}{2}\log N-\frac{t}{4}\log n}}$.
The quantity $\alpha_1(\frac{1+y-ix}{2})$ is fairly close to $\log N$ in real part. The main new difficulty is that it varies in x; but the variation is very slow and I think its contribution should be lower order. By the chain rule, the derivative of the first expression is (I think) the sum of
$\sum_{n=1}^N \frac{\frac{i}{2}\log n}{n^{(1+y-ix)/2+\frac{t}{2}\alpha_1(\frac{1+y-ix}{2})-\frac{t}{4}\log n}}$
and
$\sum_{n=1}^N \frac{-\frac{it}{2}\alpha_1'(\frac{1+y-ix}{2})\log n}{n^{(1+y-ix)/2+\frac{t}{2}\alpha_1(\frac{1+y-ix}{2})-\frac{t}{4}\log n}}$.
The first display here is similar to what we got for the derivative of B^{toy}/B_0^{toy}. Hopefully if we just take absolute values everywhere we will get a usable bound, unfortunately I can't think of any slick shortcuts to avoid doing this.
As for A^{eff}/B_0^{eff}, one may wish to factor this into A^{eff}/A_0^{eff} and A_0^{eff}/B_0^{eff}. The latter expression is a bit messy its magnitude (which is called λ in http://michaelnielsen.org/polymath1/index.php?title=Controlling_A%2BB/B_0#Analysing_the_effective_model) can be bounded in magnitude by $e^\delta N^{-y}$ where δ is given in
http://michaelnielsen.org/polymath1/index.php?title=Controlling_A%2BB/B_0#Analysing_the_effective_model . So one could work with the triangle inequality
$$\left|\frac{A^{eff}+B^{eff}}{B_0^{eff}}\right| \geq \left|\frac{B^{eff}}{B_0^{eff}}\right| - e^\delta N^{-y}\left|\frac{A^{eff}}{A_0^{eff}}\right|$$
and then we would only need derivative bounds on $\frac{A^{eff}}{A_0^{eff}}$ rather than $\frac{A^{eff}}{B_0^{eff}}$. This should be about as difficult as getting derivative bounds on $\frac{B^{eff}}{B_0^{eff}}$. Probably the bounds will be slightly worse, but on the other hand we have the factor of N^{-y} which should ultimately make the bound on this term better than for the B factor.

Terence Tao · 5 March 2018

Oh, what I was proposing was not to bound $\left|\frac{A^{eff}+B^{eff}}{B_0^{eff}}\right|$ directly by evaluation at a mesh and upper bound on first derivative (due to the messy nature of the

derivative of A^{eff}/B_0^{eff}, but rather to bound the smaller quantity $\left|B^{eff}\right|/\left|B_0^{eff}\right| - e^{\delta}N^{-y}\left|A^{eff}/A_0^{eff}\right|$, by evaluating that a mesh and getting an upper bound for the derivatives of B^{eff}/B_0^{eff} and A^{eff}/A_0^{eff}.

But perhaps what we should do first is get an approximation to the size of the derivatives involved. At any given point, one can upper bound the derivative of say B^{eff}/B_0^{eff} by evaluating the derivative of each summand $\frac{1}{n^{\frac{1-y}{2}+\frac{t}{2}\alpha_1-\frac{t}{4}\log n}}$, taking absolute values, and then summing. This removes all the oscillation and should give a bound that doesn't vary much in x; probably it will in fact be decreasing in x. So evaluating it at just one point, say the left endpoint $x = x_N = 4\pi N^2 - \pi t/4$ of the interval $[x_N, x_{N+1})$ associated to a given value of N, will give a reasonable proxy for an actual uniform upper bound on the derivative on the entire interval. Similarly for any other ratio (e.g $B^{eff} + A^{eff}/B_0^{eff}$, A^{eff}/A_0^{eff}, etc.) that we would be interested in. If these bounds are reasonable then we could just simply take a conservative fixed mesh and then try later to justify the required bounds on the first derivative across the interval. (One possibility would be to evaluate the first derivative upper bound at one point of the interval and use bounds on the derivatives of that upper bound to extend them to the rest of the interval. That may seem even more complicated, but my guess is that the derivative of the upper bound should be roughly $1/x$ of the upper bound itself (basically because all the oscillation is gone), so even with rather crude estimates this should give fairly good estimate.)



KM · 4 March 2018

Thanks. Also maybe a typo. For the derivative upper bound, should we use the reverse triangle inequality?



KM · 5 March 2018

These were some of the sample bounds I obtained for |d/dx(B_eff/B0_eff)| and |d/dx(A_eff/A0_eff)| (by taking the sum of the absolute values of the summands)

N, |d/dx(B_eff/B0_eff)| , |d/dx(A_eff/A0_eff)|

20, 1.94, 4.62

50, 2.33, 7.13

100, 2.27, 8.36

150, 2.12, 8.64

200, 1.98, 8.63

250, 1.86, 8.50

Both the bounds do seem to slowly decrease with x for a given N, although there are discontinuities at the N jumps. Eventually both decrease with N as well.

Also, the |d/dx(A_eff/A0_eff)| bound goes much higher than the |d/dx(B_eff/B0_eff)| bound.

Before proceeding, I just wanted to check about these sample estimates and the formula for d/dx(B_eff/B0_eff). For each summand, the numerator expression is coming out to be (i/2)(1 + (t/2)*alpha1′(s))*log(n)

with s as (1+y-ix)/2, which seems different from the derivation above.



Terence Tao · 5 March 2018

Oops, I had for some reason differentiated with respect to n instead of x which is of course completely wrong. I've corrected my previous comment accordingly. But it does mean that the derivative is not as complicated as I had feared.

There is some chance that the oscillation in A_0^{eff}/B_0^{eff} counteracts the oscillation in A^{eff}/A_0^{eff}, leading to a fairly good bound for the derivative of the ratio $A^{eff} + B^{eff}/B_0^{eff}$ as opposed to $\left|B^{eff}/B_0^{eff}\right| - e^{\delta}N^{-y}\left|A^{eff}/A_0^{eff}\right|$. This may be useful for very small values of x or N as the former may stay a little bit further away from the origin. I will try to calculate some usable bounds for these quantities soon.



Terence Tao · 6 March 2018

I have now computed a messy but explicit upper bound on the derivative of $\frac{A^{eff}+B^{eff}}{B_0^{eff}}$ that depends only on N at http://michaelnielsen.org/polymath1/index.php?title=Controlling_A%2BB/B_0#Derivative_bounds . I haven't checked yet how well it compares numerically with the actual derivative.
Incidentally, the wiki pages are beginning to sprawl a bit. I plan to write up a more logically ordered description of the argument, perhaps in the next blog post when hopefully we would have completed the verification of the test problem.



KM · 6 March 2018

I think in the large N case in the wiki, the 0.8 factor should be 0.08. Although the quantity in the exponential is quite small, so the calculations do not change.
[Corrected, thanks – T.]



KM · 7 March 2018

For N from 1 to 500, I was able to compute the upper bounds of |d/dx[(A_eff+B_eff)/B0_eff]| using the new formula.
Maximum value is at N=38 -> 7.1826, after which it shows a consistent decrease. The file with values for each N is also kept in the research folder.
Although these bounds seem to be about twice compared to the one obtained in the toy model case (https://terrytao.wordpress.com/2018/02/24/polymath15-fourth-thread-closing-in-on-the-test-problem/#comment-493109), they should not pose a major challenge during large scale computations.



KM · 7 March 2018

Is the 4*pi*n^2 factor in the A_eff section of the formula actually 4*pi^2*n^2? In which case, there is some improvement in the upper bound.



Terence Tao · 7 March 2018

Wow, good catch! Indeed it is even better, it is $(4\pi)^2 n^2 = 16\pi^2 n^2$. I've corrected the wiki accordingly.



KM · 7 March 2018

Thanks. I have updated the formula and the table. The maximum is now at N=45 -> 6.2573, post which there is a consistent decline.
Rudolph and I have been also attempting to calculate the |exact derivative| at any x. One nuance we wanted to check on is whether we should here use –logn * ddx(sA) – ddx(logλ) instead of –logn * ddx(sA) + ddx(logλ), since ddx(logλ) has been derived using s=(1+y-ix)/2.



Terence Tao · 7 March 2018

I think it should be the plus sign:
$\frac{d}{dx}\log(\lambda\frac{1}{n^{s_A}}) = \frac{d}{dx}(\log\lambda - (\log n)s_A) = \frac{d}{dx}\log\lambda - (\log n)\frac{d}{dx}s_A$

and hence
$\frac{d}{dx}(\lambda \frac{1}{n^{s_A}}) = \lambda \frac{1}{n^{s_A}}(-(\log n)\frac{d}{dx}s_A + \frac{d}{dx}\log\lambda)$.
Of course, by the chain rule, we then have $\frac{d}{dx}\log\lambda = -\frac{i}{2}\frac{d}{ds}\log\lambda$.
One could numerically compare the exact derivative that one computes in this fashion against a Newton quotient for $(A^{eff} + B^{eff})/B_0^{eff}$, evaluated at some moderately small spacing (e.g. $h = 0.001$) that is smaller than the expected wavelength of oscillation but larger than the numerical accuracy, as this would provide another checksum to make sure that we haven't made any further typos or freshman calculus errors somewhere.

Rudolph · 8 March 2018

KM and I have now checked and matched our results for $\left|\frac{d}{dx}\frac{A^{eff}+B^{eff}}{B_0^{eff}}\right|$ as well as for its bound. We have also numerically compared some values to the Newton quotient and do get a very good match for -logn * ddx(sA) – ddx(logλ), however for -logn * ddx(sA) + ddx(logλ) (per wiki) the numbers seem off. Could a minus sign maybe 'hide' somewhere in the definition of ddx(logλ) ?
Below are a few graphs to visualise the results. The derivative seems to stay well within its bound even for small values of x :)
$15 \leq x \leq 10^2$ https://ibb.co/eMTwq7
$100 \leq x \leq 10^3$ https://ibb.co/btbzA7
$100 \leq x \leq 10^4$ https://ibb.co/eGmo3S
$100 \leq x \leq 10^5$ https://ibb.co/dAU83S
$100 \leq x \leq 10^6$ https://ibb.co/escBOS
$100 \leq x \leq 10^7$ https://ibb.co/gxYkiS



Terence Tao · 8 March 2018

Ah, I think I identified the problem: I had written s incorrectly on the wiki as $\frac{1+y-ix}{2} = 1 - s$, when instead it is $\frac{1-y+ix}{2}$; this effectively causes λ to be incorrectly replaced by $1/\lambda$ which presumably explains the wrong sign for $\frac{d}{dx}\log\lambda$. Also I had incorrectly written the main term of $\frac{d}{dx}s_A$ as $-i/2$ instead of $i/2$. Hopefully it all matches now!



Rudolph · 7 March 2018

There is probably a small typo in the wiki about the estimation of E1, E2. These two currently show as:
$s_N^+ := \frac{1+y+i(x_N+x_{N+1})/2}{2}$
$s_N^- := \frac{1-y+i(x_N+x_{N+1})}{2}$
I guess the first display is the correct one and the /2 is just missing in the second one? Impact on the overall result is minor.



Terence Tao · 7 March 2018

Yes, this was a typo, now corrected. Thanks for pointing it out.



VGM Kin · 2 March 2018

Hi Professor Tao, I read a paper claiming that there exists general solution to equations of fifth degree and above which contradicts with Galois group theory. What's your view on that?
The General Quintic Equation, its Solution by Factorization into Cubic and Quadratic Factors
http://www.rroij.com/open-access/pdfdownload.php?download=open-access/the-general-quintic-equation-its-solution-by-factorization-into-cubicand-quadratic-factors-.pdf&aid=86295

Anonymous · 3 March 2018

I'm not Prof. Tao but that paper sounds not much different from an angle trisection construction or a claim that two even numbers can add up to an odd number. You can tell it's wrong just from what it claims.

Anonymous · 3 March 2018

In the wiki page http://michaelnielsen.org/polymath1/index.php?title=Controlling_H_t-A-B/B_0 what values of t and y are used to generate the tables? Are the test problem values t=y=0.4 used for all tables?

David Bernier (@doubledeckerpot) · 3 March 2018

For the tables with x ranging from 160 to 3200, by increments of 160, I set y = 0 , and t = 0.4 .
For the table that follows, showing a "spike" or discontinuity near x = 804, I think I also set t = 0.4, y=0.

David Bernier (@doubledeckerpot) · 3 March 2018

I recalculated $(H^{eff}(z) - (A(z) + B(z) - C(z))/B_0(z)$, in absolute value, but this time with $t = y = 0.4$.
I'd say it's roughly comparable to the $y = 0$ case.
? for(K=1, 5, x50= 160*K+0.4*I ;s=0.0;for(J=0,40,s=s+intnumgauss(X=J/14,(J+1)/14, exp(t*X*X)*Phi(X)*cos((x50)*X),)); s2=Heff2(x50); b0=B0(x50);s4 = abs((s2-s)/b0);w20[K] = s4;)
? for(K=1, 5, print(160*K," ", w20[K]))
160 0.00708556855789
320 0.00243908316749
480 0.000342587313046
640 0.00107187454359
800 0.00185702885238

Anonymous · 3 March 2018

The test problem is to show that $H_t(x + iy) \neq 0$ whenever $t = y = 0.4$ so it might be helpful to people joining the project if we had some reference evaluations of this function for a few values of x. I couldn't find this on the wiki or in blog posts for this project (although I would be surprised if I did not simply overlook it) so I'll write some down here, to be cross-checked by others I hope.
$H_{0.4}(10 + 0.4i) = 0.034420270187052311229 - 0.0016782531784355935738i$
$H_{0.4}(30 + 0.4i) = -0.00010001026469315639165 - 7.1357019921469872653 \times 10^{-6}i$
$H_{0.4}(100 + 0.4i) = 6.7021522172791266841 \times 10^{-16} + 3.1337965840705569244 \times 10^{-16}i$
$H_{0.4}(300 + 0.4i) = -4.0159674206251463625 \times 10^{-49} - 1.4006524430296850032 \times 10^{-49}i$
$H_{0.4}(1000 + 0.4i) = 1.4847586783170623283 \times 10^{-169} + 3.0506306235580557897 \times 10^{-167}i$
$H_{0.4}(3000 + 0.4i) = -1.1441895900436789748 \times 10^{-507} + 1.5701563504659934332 \times 10^{-507}i$
$H_{0.4}(10000 + 0.4i) = -5.5773401535231430494 \times 10^{-1701} - 4.0878432123908616412 \times 10^{-1700}i$

Terence Tao · 3 March 2018

Thanks for this! I put this data, together with the various approximations we have for H_t at these values of x. The accuracy of approximation is pretty terrible for small values of x (and for $x = 10$ we in fact have no approximation at all because $N = 0$ and several terms degenerate), but they all improve as x increases and our most advanced approximation $A^{eff} + B^{eff} - C^{eff}$ does surprisingly well even for x as small as 30. [EDIT: actually it even does passably well for $x = 10$!]



David Bernier (@doubledeckerpot) · 5 March 2018

I got the same values for $H_{0.4}(10+0.4i)$ to $H_{0.4}(3000+0.4i)$, using the integral in PARI/gp. The computation of $H_{0.4}(10000+0.4i)$ isn't finished.



Rudolph · 8 March 2018

Got some more exact Ht values for lower x. Using the integral, I have computed Ht at 40 digits accuracy for $x = 0...300$ in steps of 0.1. As usual $t = y = 0.4$. The three files can be found on the link below. I can confirm an exact match with the x=10, 30, 100 and 300 mentioned above.
https://github.com/km-git-acc/dbn_upper_bound/tree/master/dbn_upper_bound/python/research/H_t%20at%20small%20x



Terence Tao · 8 March 2018

Thanks for this! To complete the verification of a zero-free region here, we will need derivative bounds on H_t. I think I know a way to do this using integration by parts. Will write details later, but for now: using the notation of http://michaelnielsen.org/polymath1/index.php?title=Effective_bounds_on_H_t_-_second_approach , we have $H_t(z) = \frac{1}{8}\xi_t(s)$, where $s = \frac{1+iz}{2}$ and ξ_t has an expansion

$$\xi_t(s) = \sum_{n=1}^{N} F_{t,n}(s) + F_{t,n}(1-s) + G_{t,N}(s) + G_{t,N}(1-s)$$

where $F_{t,n}$ in turn has an integral representation

$$F_{t,n}(s) = \exp(-\frac{t}{4}\alpha_n^2)\int_{-\infty}^{\infty} \exp(-\sqrt{t}\alpha_n u) F_{0,n}(s+\sqrt{t}u+\frac{t}{2}\alpha_n)\frac{1}{\sqrt{\pi}}e^{-u^2}\,du$$

with a similar formula for $G_{t,N}$.

Now by the chain rule we have $\frac{d}{dz}H_t(z) = \frac{i}{16}\frac{d}{ds}\xi_t(s)$, which in turn expands to a sum of terms such as $\frac{d}{ds}F_{t,n}(s)$. If we differentiate the above formula by s (presumably we can justify differentiation under the integral sign as everything is analytic and has good decay at infinity) we obtain

$$\frac{d}{ds}F_{t,n}(s) = \exp(-\frac{t}{4}\alpha_n^2)\int_{-\infty}^{\infty} \exp(-\sqrt{t}\alpha_n u)\frac{d}{ds}F_{0,n}(s+\sqrt{t}u+\frac{t}{2}\alpha_n)\frac{1}{\sqrt{\pi}}e^{-u^2}\,du.$$

One can replace $\frac{d}{ds}$ here by $\frac{1}{\sqrt{t}}\frac{\partial}{\partial u}$ and integrate by parts and eventually end up at

$$\frac{d}{ds}F_{t,n}(s) = \alpha_n F_{t,n}(s) + \exp(-\frac{t}{4}\alpha_n^2)\int_{-\infty}^{\infty} \exp(-\sqrt{t}\alpha_n u)F_{0,n}(s+$$
$$\sqrt{t}u+\frac{t}{2}\alpha_n)\frac{1}{\sqrt{\pi}}2ue^{-u^2}\,du.$$

One can upper bound this using the techniques on the wiki, and similarly for the other terms in the expansion for ξ_t. The upshot should be that we get bounds for $\frac{d}{dz}H_t(z)$ that are not much larger than H_t itself (it's looking like it will be about $O(\log T)$ larger, which is what one should expect given that H_t oscillates at scale $1/\log T$ as one already sees from the asymptotics of zeroes). I'll try to work out an explicit bound on wiki soon.



Anonymous · 8 March 2018

It seems that by repeating this differentiation trick, it is possible to get similar expression (with extra factor $O(\log T)$) for the second derivative (and higher derivatives)



Anonymous · 9 March 2018

Is there any connection to Bernstein's inequality for the derivative of a trigonometrical polynomial?

Anonymous · 9 March 2018

More evaluations of Ht with t=y=0.4:
x=200 to x=600 with step size 0.1: https://pastebin.com/fim2swFu
x=600 to x=1000 with step size 0.1: https://pastebin.com/jvSvDP69
x=30000:
$3.1592485183221675757 \times 10^{-5110} - 6.7370201647463853979 \times 10^{-5110} i$

> **Terence Tao** · 10 March 2018
>
> Thanks for this! I have put the data on the wiki at the bottom of http://michaelnielsen.org/polymath1/index.php?title=Polymath15_test_problem . If the plots of this data (perhaps normalised by dividing by B_0^{eff} look smooth (in particular having enough resolution to capturing each wavelength of the oscillation of H_t) then there is a good chance that one can establish analytic upper bounds on the first derivative of H_t to fill in the gaps between the step size 0.1 mesh.
>
> > **Rudolph** · 11 March 2018
> >
> > Below is a link to some 'snapshot' scatter plots to visually inspect the behaviour of Ht^{act} (normalised by $\left|B0^{eff}\right|$). It appears that the wavelengths of the oscillations of Ht^{act} in this domain are way larger than 0.1, so it looks good :)
> > https://ibb.co/fOroa7

Rudolph · 11 March 2018

Your evaluations at such large x are really impressive, Anonymous. I actually start to struggle, both in terms of computation time and accuracy required, for $x \geq 1000$. Would you be willing to share how you performed these exact computations?

> **Anonymous** · 11 March 2018
>
> Rudolph I used the plain integral of Φ formula, with the tail errors of the integral and sum bounded by the values derived in https://terrytao.wordpress.com/2018/03/02/polymath15-fifth-thread-finishing-off-the-test-problem/#comment-493653. This is a crude way to compute Ht because of the oscillation, and I hope someone can help to derive similar tail error bounds for the sum K and integral I that are defined in the first Polymath15 thread https://terrytao.wordpress.com/2018/01/27/polymath15-first-thread-computing-h_t-asymptotics-and-dynamics-of-zeroes/ so that Ht can be computed faster and more accurately if more Ht values are still needed for this project.
>
> > **Terence Tao** · 11 March 2018
> >
> > I'll look into this. I'm finding that the existing techniques to control H_t and its first derivative deteriorate as x (or T) become relatively small (e.g. $x \sim 100$) for a number of reasons, including the fact that one is getting a bit too close to the singularities of the Gamma function. This is unfortunately because this is the region where we really need derivative bounds! So one may indeed have to go back to earlier ways to compute H_t to bound things. A model problem would be to get an effective bound of the shape $H_t(x+iy) \ll x^{O(1)} e^{-\pi x/8}$ that works for smallish x, e.g. $0 \leq x \leq 100$; I know that this is what happens asymptotically (in fact $H_t(x+iy)$ is roughly like $x^{\frac{7+y}{4}} e^{-\pi x/8}$) but as mentioned above the arguments (based on the fundamental solution to the heat equation) lose potency for small x. A naive triangle inequality estimate will give some bound on $H_t(x+iy)$, but it's likely to be useless if it doesn't have the $e^{-\pi x/8}$ decay

factor. Perhaps contour shifting the original definition of H_t will do the trick; I'll think about it.

 **Terence Tao** · 12 March 2018

OK, I've written some estimates on both $H_t(x+iy)$ and $H_t'(x+iy)$ at http://michaelnielsen.org/polymath1/index.php?title=Bounding_the_derivative_of_H_t_-_second_approach that should be amenable for numerical computation at low values of x. As discussed in an early blog post, there is a splitting
$H_t(x+iy) = \frac{1}{2}\sum_{n=1}^{\infty} 2\pi^2 n^4 I_{t,\theta}(x-(9-y)i, \pi n^2) - 3\pi n^2 I_{t,\theta}(x-(5-y)i, \pi n^2)$
$+2\pi^2 n^4 I_{t,\theta}(x-(9+y)i, \pi n^2) - 3\pi n^2 I_{t,\theta}(x-(5+y)i, \pi n^2)$
where $0 < \theta < \pi/8$ is a parameter one is free to choose, and $I_{t,\theta}(b,\beta)$ are certain oscillatory integrals. It seems that a good choice to reduce oscillation is $\theta = \frac{\pi}{8} - \arctan\frac{9+y}{x}$. On the wiki there are tail bounds for each individual $I_{t,\theta}(b,\beta)$ that should allow one to cut off each individual integral at some reasonable cutoff (something like a large multiple of $\log x$ should work) with an exponentially decaying bound on the tail, and then there is also a bound for the tail of the sum when n is large (something like a large multiple of $\sqrt{x/4\pi}$ should work). There is also a similar analysis for the derivative $H_t'(x+iy)$. One should first test that these estimates are actually effective for evaluating $H_t(x+iy)$ and then look at how they bound $H_t'(x+iy)$, hopefully they can get good uniform bounds on intervals such as $x \in [x_0, x_0+0.1]$ which is the mesh size we are dealing with.

 **Anonymous** · 11 March 2018

For very small values like x < 20 it is possible to use brute force interval computation of the original Ht formula on a reasonably sized mesh to bound Ht away from zero. If the analysis of the derivative bounds of Ht breaks down for x in this range then the interval approach may be a practical although less elegant option.
As x becomes larger like between 30 and 100, this brute force computation with intervals begins to require unreasonably sized meshes, although I suppose it could be done that way if absolutely necessary.
If the rearranged Ht formula that uses K and I improves numerical stability then I would be surprised if it doesn't also extend the range of x for which brute force interval computation allows Ht to be bounded away from zero with a mesh of practical size. Maybe it could reach x=100. I don't know. I hadn't tried that formula yet because I don't have truncation error bounds for the series or the integral, but maybe I will give it a closer look.

 **Anonymous** · 11 March 2018

For smaller t values, it seems that H_t asymptotic approximations would become less efficient (requiring larger x values) such that for sufficiently small t and moderately large x, the current approximation methods might become impractical.

 **David Bernier (@doubledeckerpot)** · 12 March 2018

I agree with Rudolph, there's a lot of computation involved. Would you mind sharing what software package you use to compute the integral with the function Φ ?

 **Rudolph** · 12 March 2018

Maybe a small typo in the first display under "or any cutoff X≥0, where". Shouldn't it read $I_{t,\theta,\leq X}(b,\beta)$?

[Corrected, thanks. Also corrected some missing factors of $2tX$ in some of the formulae. -T.]

Rudolph · 15 March 2018

Found two more small typo's on the "second approach"-wiki.
Under the header "and by arguing as before" in the RHS of the inequality, πn^2 should be πn_0^2 and $\cos\theta$ should be $\cos 4\theta$. Under "and similarly" also πn^2 should become πn_0^2 in the RHS.
[Corrected, thanks – T.]

KM · 15 March 2018

To update a bit on the ongoing effort for small x, we now have the H_t integral estimates and the A+B-C estimates staying close to each other, and similarly for the H'_t and newton quotient estimates, indicating the estimates are being generated correctly.
The integral estimates require setting increasing amounts of precision as x is increased, which slows things down. For eg. at x=1500, the single point evaluation of Ht took me around 13 minutes. @Anonymous, it would be cool to know your tool or optimization techniques given you had generated many estimates around these x magnitudes.
I seem to have got a |H't|/|B0| bound of 6.85 using the two triangle inequalities at x=20 (and the corresponding theta). Evaluating exact |H't|/|B0| at a few hundred points, the values seem to stay sufficiently below the bound value, but it needs to be double checked.

KM · 16 March 2018

Prof. Tao, it seems my earlier attempt to find the upper bound of H'(t) was not correct, and I have attempted again (also in the wiki, the triangle inequality for J with limit 0 to X has a cos(theta) factor, which looks like it should be cos(4*theta) and I used the latter here.
Using a constant theta_c (and large n0 and X), since the bound on J varies as D * exp(-theta_c * x), with D being a constant, which decreases less rapidly than |B0(x)|, the earlier approach of using f(x) = H_t/B0 results in rapidly decreasing mesh gaps.
It seems then we should let f(x) = H_t and then the adaptive mesh gap (without the c term) would be |H_t(x)| * exp(theta_c * x)/D. Also, using theta_c closer to 0 still results in too small mesh gaps.
Using theta_c as Pi/8 – arctan(9.4/1600) results in theta_c = 0.3868.. and D = 5969.080.. and then using c = c0 * exp(-theta_c*x) (with c0 for example 0.02, or lower if the gaps become negative), we can compute the mesh gaps as (|H_t(x)| * exp(theta_c * x) – 0.02)/D,
where the gaps now seem to stay reasonable if occasionally a bit small (0.0002 to 0.165 (evaluated at around 1600 points)). Is this approach correct?

KM · 16 March 2018

When mentioning the relationship between exp(-theta_c*x) and |B0(x)|, since whether the ratio is increasing or decreasing with x depends on theta_c, possibly it would be better to say they don't tend to keep the same order of magnitude.



Terence Tao · 16 March 2018

Sorry about the typos in the wiki – all the appearances of $\cos\theta$ have now been corrected to $\cos 4\theta$.

We won't need the factor c any more in the adaptive mesh (at least for the toy problem of showing that H_t is non-zero) because there is no longer any error term $E_1 + E_2 + E_3$ that one has to beat. So if one has a derivative bound of $De^{-\theta_c x}$ for all $x \geq x_0$ and one is currently at some $x_i \geq x_0$, then one can select the next mesh point as $x_i + |H_t(x_i + iy)|/(De^{-\theta_c x_i})$ as you indicate.

I wonder if it is possible to see how these upper bounds on $|H_t'(x+iy)|$ compare with the true value of $|H_t'(x+iy)|$ (or proxies for this, such as the derivative of $A^{eff} + B^{eff}$, if this is easier to compute). The fact that the mesh size needed is currently somewhat poor suggests that there may be room for improvement in the estimate. Also, I was hoping that by shifting the contour as on the wiki one could compute $H_t(x+iy)$ faster than we currently are doing – a 10-fold speedup in the evaluation of each $H_t(x+iy)$ is roughly equivalent in performance to a 10-fold improvement in the upper bound for $|H_t'|$.



KM · 17 March 2018

On checking the match between the exact |H't| and the newton quotient of (A+B-C), we find a good match; for example,

x abs(H't/NQ_ABC)

20 0.991

30 1.067

100 1.004

500 1.003

so I used the quotient as a proxy for the derivative below.

The ratio R = abs(D*exp(-theta_c * x)/NQ_ABC) has a large range (evaluated for each (D,theta_c) pair at around 16000 points between x = 20 and 1600). There seems to be some theta_c (corresponding to x_c between 1000 and 2000) at which the variance is smallest.

theta_c D min R max R stdev R

Pi/8 – atan(9.4/50) 0.18835 2.1 2.24E+18 2.32E+17

Pi/8 – atan(9.4/500) 222.823 5.0 7.49E+10 6.41E+09

Pi/8 – atan(9.4/1000) 1592.94 7.2 181906 19500.1

Pi/8 – atan(9.4/1600) 5969.08 9.0 6377 371.4

Pi/8 – atan(9.4/2000) 11146.7 10.3 11484 385.6

Pi/8 – atan(9.4/5000) 143062 15.7 135087 4505.5



Terence Tao · 17 March 2018

Thanks for this! I guess the reciprocal 1/R is a slightly more useful statistic than R because R is distorted by those occasions where the oscillations of NQ_ABC bring it close to the origin. But if I understand your data correctly, it seems that even in the worst case, our upper bound for H'_t is about a factor of 15 or so worse than the true size of H'_t, which basically means that we need a resolution which is something like 15 times finer than the oscillation wavelength of H_t to be able to rigorously control things.

It may be the case that the theoretical choice $\theta_c = \pi/8 - \arctan \frac{9+y}{x}$ for θ may not quite be the optimal choice numerically. One could perhaps try to assemble a collection of upper bounds $D(\theta) \exp^{-\theta x}$ for various θ (bearing in mind that each such bound may require also a lower bound $x \geq x(\theta)$) and try to locate the minimum of these bounds for various given θ to come up with a numeric rule of thumb for what the optimal θ

is for each x (or simply store all the upper bounds and work with the envelope of all of them).
A more advanced possibility is to work with analytic upper bounds on the second derivatives rather than the first, and work with numerical evaluations of the first derivative at mesh points, combined with maybe a second order Taylor expansion. This looks to be the more complicated option (in particular, more prone to bugs and typos) and probably should be kept as something of a last resort. (But in principle, given that we can sample H_t at a resolution significantly lower than its wavelength oscillation, each higher derivative used should lead to higher accuracy.)

 **Terence Tao** · 17 March 2018

By the way, there is a slight improvement to the upper bound on $J_{t,\theta,\leq X}$ in the wiki; one can replace the factor $\theta + u$ by $|i\theta + u| = \sqrt{\theta^2 + u^2}$. This is unlikely to lead to any dramatic gain though (a factor of $\sqrt{2}$ at best).

 **KM** · 17 March 2018

Also, we have been able to speed up computations significantly using in built optimizations in the numerical integration procedures. However from a formula perspective, the rate determining parameter is n0 (currently default value of 6 is being used), and possibly we have been too conservative to keep the tail estimates small.

 **KM** · 17 March 2018

After experimenting with a set of theta values and x points, it seems the minimum of the derivative bound (older bound) for x occurs at theta_c = Pi/8 – atan(9.4/x_c) where x/x_c generally falls between 1/3 and 1/4.
If we take x_c = Pi*x, and evaluate |H_t(x)|/(D(theta_c(x))*exp(-theta_c(x)*x)) at some sample x points upto 1600, we find these mesh gaps are on average around 0.04, with the minimum mesh gap slightly below 0.01, which is quite workable. Can we go ahead with such a derivative bound?

 **Terence Tao** · 17 March 2018

Looks promising! I don't have a theoretical explanation yet as to why the optimal value of θ was about a factor of 3 closer to $\pi/8$ than the stationary phase prediction of $\frac{\pi}{8} - \arctan\frac{9+y}{x}$, but I will think about it. In the meantime, we can go ahead and use this choice of θ and the mesh step size indicated and hopefully this lets us cover the numeric ranges needed. (It does seem remarkable that the initial choice of parameters $t = y = 0.4$ ends up landing quite close to the limit to what we can easily verify numerically – looks like we're not going to push much beyond 0.48, though we should certainly give it a shot once we perfect our methodology.)

 **Terence Tao** · 18 March 2018

Aha! I tracked down my error. The stationary point is at $\frac{\pi}{8} - \frac{1}{4}\arctan\frac{9+y}{x}$, not $\frac{\pi}{8} - \arctan\frac{9+y}{x}$ as previously claimed on wiki (now fixed). This I think explains the discrepancy reasonably well, and perhaps also allows us to treat small x, e.g. $x \leq 30$, without having the problem of θ becoming negative for no good reason.

 **KM** · 18 March 2018

Thanks. We have started running the adaptive mesh. The optimized J bound is around 1.33 times smaller than earlier, helping increase the mesh gap accordingly.

Anonymous · 11 March 2018

Two more batches of evaluations of Ht at t=y=0.4.
x=1000 to x=1300 with step size 0.1 : https://pastebin.com/NkFKs3pB
x=1300 to x=1600 with step size 0.1 : https://pastebin.com/k7vC6e7n
[Added to wiki, thanks – T.]

rudolph01 · 18 March 2018

With 16,000 precise datapoints available for H_t^{act} covering $x = 0...1600$ in steps of 0.1, I couldn't resist plotting a quick visual to help loosely verify whether:
$\frac{|H_t^{act}-A^{eff}-B^{eff}|}{|B_0^{eff}|} \leq \frac{|E_1|+|E_2|+|E_3^*|}{|B_0^{eff}|}$
would still hold in this domain and it seems to all work pretty well :)
https://ibb.co/n9Gw2x
https://ibb.co/cRTLUc
[Added to wiki, thanks – T.]

rudolph01 · 19 March 2018

Just spotted that I had used C^{eff} as a proxy for E_3^* in my code for the plots... Although it gives a nice picture, it is not very informative and certainly not helpful for the wiki page. Here is a corrected plot, now using the vwf-integral to estimate E3 giving a much smoother overall error term and it now also properly compares:
$\frac{|H_t^{act}-A^{eff}-B^{eff}|}{|B_0^{eff}|} \leq \frac{|E_1|+|E_2|+|E_3|}{|B_0^{eff}|}$
https://ibb.co/b7baZc
[Link corrected, thanks. I was wondering why the fit was so excellent... -T.]

rudolph01 · 18 March 2018

Below are links to exact data (40 digit accuracy) for $H_{0.4}^{'}(x+0.4i)$ covering $0 \leq x \leq 300$ in steps of 0.1:
$0 \leq x \leq 100$: https://drive.google.com/open?id=1mtzrJ_-hBMyt90gVzJdNCq4RU6Pz_dQQ
$100.1 \leq x \leq 200$: https://drive.google.com/open?id=1wSZJgoPp9-6C8CJqLr5g3E1uhEBHKmTJ
$200.1 \leq x \leq 300$: https://drive.google.com/open?id=1q3h0x8jSF0Z1KQ9iLx23JI1oCXfrx0RL
Since the "second"-integral is (currently) difficult to control for $x \leq 10$, attached is for $0 \leq x \leq 30$ in steps of 0.01 a dataset with 20-digit accurate values for the following comma-separated fields ($y = t = 0.4$):
x, pari/gp prec, H_t, $H_t^{'}$, $|H_t|$, $\left|H_t^{'}\right|$, $\frac{|H_t|}{|B_0^{eff}|}$, $\frac{\left|H_t^{'}\right|}{|B_0^{eff}|}$
https://drive.google.com/open?id=1ge0TD5hvs1O6BKLAz34JmTqxSrguvjsJ
[Links added to wiki, thanks -T.]

Anonymous · 17 March 2018

At https://pastebin.com/ThDzc4bn I've made available a more nearly self-contained program from the code that I had been using to compute Ht. It requires a library called arb which can be found at http://arblib.org/.

David Bernier (@doubledeckerpot) · 18 March 2018

I've got it working up to x = 2000, but at x = 3000, it says "core dumped"...
This is the data output for 1000 and 2000:

$ time ./a.out 0.4 1000 0.4 20
Re[H]: 1.4847586783170623283e-169
Im[H]: 3.0506306235580557897e-167
real 0m1.540s // 1.5 seconds
and
$ time ./a.out 0.4 2000 0.4 20
Re[H]: -5.9377117202337490654e-337
Im[H]: -2.0564217656272998761e-337
real 0m17.648s
It seems very fast.

Anonymous · 18 March 2018

Thanks, I see that it aborted (not segfault) because its integral didn't numerically converge. I'll look into why it does that. Maybe I made some errors when I converted my pile of code into a single C file, or maybe the ad hoc way of increasing precision is too ambitious for the absolute error tolerance target that I'm giving the integration routine or something.

Anonymous · 18 March 2018

I found the cause, and a workaround is that you can change prec=8 in the code to prec=500 or something like that. I'll work on a better solution.



David Bernier (@doubledeckerpot) · 18 March 2018

Ok, thanks! The values I got for x = 10 , 30, 100, 300 and 1000 agree with those given by someone that goes by Anonymous.

Anonymous · 18 March 2018

Here's an updated code where I've changed how the working precision is initialized and removed the hard abort if an integral doesn't converge at a given working precision https://pastebin.com/QDPvWa3g. I checked it at x=3000 and x=10000. By the way, your timings are similar to the ones on my system.

Anonymous · 23 March 2018

Line 36: should arb_get_mag be arb_get_mag_lower?

Anonymous · 5 March 2018

Is something like interval analysis should be required at some point for a rigorous treatment of numerical rounding errors?



Terence Tao · 5 March 2018

I guess so, but I think the main priority would be to get to the point where the numerical verifications are routine enough (and have enough "room" that one does not need extreme amounts of numerical precision) that they can be replicated in reasonable time (e.g. less than a day) by multiple computer software packages, including those that use interval arithmetic. Given that the numerical conclusions of this project are unlikely to be crucial in future research work (in contrast to, say, numerical work on RH, which is often used elsewhere in effective number theory), I think this standard of replicability will be sufficient for publication purposes, though there is of course always the possibility that the referees may disagree.

Anonymous · 5 March 2018

Thank you for hosting this polymath! I have a couple of embarrassingly simple questions related to naive evaluation of Ht and Phi from their definitions at the top of the wiki page http://michaelnielsen.org/polymath1/index.php?title=De_Bruijn-Newman_constant

Phi is represented as a sum from n=1 to n=infinity, and I would like to know an explicit but possibly loose upper bound of the size of the sum from say n=k to n=infinity. The formula for this remainder bound would be a function of u and k, and I would like it to hold for complex u when -pi/8 < Im(u) < pi/8 and when the real part of u is unrestricted so that it can be either positive or negative or zero.

Ht is represented as an improper integral from u=0 to u=infinity, and I'd also like to know an upper bound for the size of the remainder of this integral. In other words, I'd like to know an upper bound for the absolute value of the integral from u=a to u=infinity. This bound would be a function of z, t, and a.

I know that this probably isn't the best way to numerically evaluate Ht, and I also know that you can probably just take k = a = 100 and the remainder would be negligible, but I would still like to know explicit bounds for these remainders.

> **Terence Tao** · 5 March 2018
>
> I don't know if anyone has explicitly computed bounds for the tail of Φ here, but given the rapid decay, perhaps even very crude bounds would be sufficient, as long as $\mathrm{Im}(u)$ stays a little bit away from $\pm\pi/8$, say $\mathrm{Im}(u) \leq \frac{\pi}{8} - \varepsilon$ so that e^{4u} has some substantial real part in the sense that its argument is between $-\frac{\pi}{4} + 4\varepsilon$ and $\frac{\pi}{4} - 4\varepsilon$, so that $\left|e^{4u}\right| \leq \mathrm{cosec}(4\varepsilon)\mathrm{Re}e^{4u}$. If for $\theta = 5/4, 9/4$ one lets C_θ be a constant for which $x^\theta \leq C\exp(x)$ for all $C > 0$, then
> $2\pi^2 n^4 \left|e^{9u}\right| \leq \mathrm{cosec}(4\varepsilon)^{9/4} 2\pi^2 (\pi/2)^{-9/4} (\pi n^2 \mathrm{Re}(e^{4u})/2)^{9/4}$
> $\leq \mathrm{cosec}(4\varepsilon)^{9/4} 2\pi^2 (\pi/2)^{-9/4} C_{9/4} \exp(\pi n^2 \mathrm{Re}(e^{4u})/2)$
> and similarly for $3\pi n^2 \left|e^{5u}\right|$, which should allow one to bound each summand in Φ by an explicit multiple of $\exp(-\pi n^2 \mathrm{Re}e^{4u}/2)$ in magnitude. One can then control the tail of such a sum $\sum_{n \geq n_0} \exp(-\pi n^2 \mathrm{Re}e^{4u}/2)$ by for instance comparing it with a geometric series (e.g. replacing n^2 with nn_0). Presumably these sorts of bounds can also then be used to control the tail of H_t.
>
> > **Anonymous** · 6 March 2018
> >
> > Thanks for the response! After reading it carefully I'm still slightly confused on some small points. I assume that "one lets C_σ" should be "one lets C_θ and that this is just a typo. Next I think that "for all $C > 0$" should be "for all $x > 0$ but I'm not as sure. Later in the main inequality chain, I think that $\left|\exp(\pi n^2\ \mathrm{Re}(\exp^{4u})/2)\right|$ shouldn't need the absolute value because it's the exponential of a real value. Maybe it's written this way on purpose and it doesn't matter that it's redundant, but I don't know. Finally I think that "explicit multiple of $\exp(-\pi n^2\ \mathrm{Re}\exp^{4u})$" should be "explicit multiple of $\exp(-(1/2)\pi n^2\ \mathrm{Re}\exp^{4u})$", but this is the part I'm least sure about and is my main point of confusion.
> >
> > > **Terence Tao** · 7 March 2018
> > >
> > > Thanks for the corrections! Another slightly different way to proceed is just to bound n^2 by nn_0 from the beginning and use the generalised geometric series formulae to evaluate expressions such as $\sum_{n \geq n_0} 2\pi^2 n^4 \exp(-\pi n n_0 \mathrm{Re}e^{4u})$ explicitly.
>
> **Anonymous** · 8 March 2018
>
> I've followed this procedure for both the series remainder and the integral remainder.
> For the series remainder I began by choosing eps=pi/24 so that pi/8 − eps = pi/12 and cosec(4 eps)=2. Then I set $C_{9/4} = 1$ and $C_{5/4} = 1/2$ and checked that $x^{9/4} < e^x$ and $x^{5/4}0$.

This gives a constant coefficient $(2\pi^2 + (3/2)\pi)2^{9/4}(\pi/2)^{-9/4} < 50$ as a multiple for each summand. Summing the geometric series $\sum_{n \geq n_0} 50\exp(-\pi n n_0 \mathrm{Re}(e^{4u})/2)$ gives the upper bound $\frac{50\exp(-\pi n_0^2 \mathrm{Re}(e^{4u})/2)}{1-\exp(-\pi n_0 \mathrm{Re}(e^{4u})/2)}$.
In preparation for the integral remainder we can find a more convenient bound for the series remainder when $n_0 = 1$. Using the inequality $1/(\exp(\pi x/2 - 1) < 2\exp(\pi x/2)$ for any $x \ge 1$ we have the bound $50/(\exp(\frac{\pi}{2}\mathrm{Re}(e^{4u}) - 1)) < 100\exp(-\frac{\pi}{2}\mathrm{Re}(e^{4u}))$.
This lets us bound the integral remainder by $100\int_b^\infty e^{tu^2}\exp(-\frac{\pi}{2}e^{4u})\cos(zu)du$. For our test problem we can let $t < 1/2$ and $|\mathrm{Im}(z)| < 1/2$, so the integrand is bounded by $\exp(u(u+1)/2 - \frac{\pi}{2}e^{4u})$ which is less than $\exp(4u - e^{4u})$ when $u \ge 0$. This gives an integral remainder upper bound of $100\int_b^\infty \exp(4u - e^{4u})du = 25e^{-e^{4b}}$. A consequence of this bound would be that in the direct evaluation of $H_{0.4}(z)$ for our test problem, approximating the improper integral $\int_0^\infty$ by the incomplete integral $\int_0^3$ gives an absolute error of less than 10^{-70000}.

Anonymous · 5 March 2018

In the wiki page "controlling $H_t - A - B/B_0$" there are two unprocessed displayed formulas. *[Fixed, hopefully -T.]*

Anonymous · 7 March 2018

Is it possible to deduce (for any $t > 0$) from the effective approximation on $H_t(z)$, a lower bound of the form $C_t/\log T$ on the horizontal gap between any two zeros of H_t with real parts greater than T? If true, this should limit (via H_t zero dynamics) the horizontal velocities of the zeros to about $O(\log T)$.



Terence Tao · 7 March 2018

This should be the case (and is consistent with Theorem 1.4 of Ki-Kim-Lee), provided T is large enough depending on t (basically one needs $t \log T$ to be sufficiently small); see also the bottom of http://michaelnielsen.org/polymath1/index.php?title=Asymptotics_of_H_t . This indeed keeps the dynamics stable for large values of t. However the problem is with the very small values of t, e.g. $0 \leq t \leq 10^{-10}$, in which the Ki-Kim-Lee asymptotics (or the ones in our project) are largely unusable. This epoch is somewhat analogous to the period shortly after the Big Bang in cosmology: while we understand pretty well what happens after this period, the dynamics in this period are really quite unclear, and in particular I don't have a good way of preventing a huge number of zeroes coming in from infinity to settle into small and medium values of T during this period, other than to start evaluating H_t for larger values of t to verify that such zeroes are no longer present.

Anonymous · 7 March 2018

It seems that there is no clear numerical threshold obstruction (i.e. "very small" t) for further analytical(!) progress in our understanding of the minimal distances among H_t zeros.



Terence Tao · 7 March 2018

I've written some notes on the wiki (here and here) indicating how to finish off the case of very large x (in which $N \geq 2000$). Here there is a fair amount of room and even relatively crude estimates work. In particular, from the triangle inequality one can bound $\left|\frac{A^{eff}+B^{eff}}{B_0^{eff}}\right| \geq 0.128$ and from crude (and somewhat inelegant and inefficient) bounds on error terms one can bound $\left|E_1/B_0^{eff}\right| \leq 2.8 \times 10^{-8}$, $\left|E_2/B_0^{eff}\right| \leq 2.9 \times 10^{-7}$, and $\left|E_3^*/B_0^{eff}\right| \leq 1.6 \times 10^{-5}$ (this follows from this table and the

monotone nature of this ratio), so there is plenty of room to keep H_t/B_0^{eff} away from zero here (indeed even the real part stays positive).
Presumably by using Euler factor mollifiers and the lemma refining the triangle inequality, together with the table on E_1, E_2, E_3^* bounds, one can cover all N between roughly 300 and 2000, leaving only the $N \leq 300$ case to be verified numerically.

KM · 8 March 2018

On evaluating the exact |derivative| (matched with the newton quotient) of the effective f = (A+B)/B0 at a few thousand points, we see that the upper bound is reasonably sharp, and the derivative has a tendency to jump towards the bound from time to time. The average derivative is much lower.
So we will start with the large scale adaptive mesh calculations using x_next = x + |f(x)|/max|f'(x)|

Terence Tao · 8 March 2018

KM, one thing is that using the adaptive mesh $x_{next} = x + \frac{|f(x)|}{\max|f'(x)|}$ will only let one verify that $|f(x)| > 0$ throughout the interval. If one wants to instead prove a lower bound of the form $|f(x)| > c$ throughout an interval, one will need to use the finer mesh $x_{next_*} = x + \frac{|f(x)|-c}{\max|f'(x)|}$. Here presumably one should take c to be the upper bound for $\frac{E_1+E_2+E_3}{|B_0^{eff}|}$ that one has from the tables in the repository.

KM · 8 March 2018

Thanks. I will use the finer mesh as you outlined.

Terence Tao · 8 March 2018

One final thing: you should set the mesh to halt if $|f(x)| \leq c$, since otherwise the mesh will go backwards or stay still and one may end up with an infinite loop (or wander beyond the left edge of the interval). In fact one may wish to call for a halt if $|f(x)| \leq c + \varepsilon$ for a very small ε.

KM · 9 March 2018

Since for N>=20, maximum |overall error/B0| was around 0.0647, I have set c to a constant 0.065 to simplify things. This takes care of the small epsilon factor as well for the halting part.

KM · 10 March 2018

The mesh exercise for N=151 to 300, y=0.4, t=0.4, c=0.065 is complete. There were around 4.4 mil mesh points. I have kept the file (somewhat large) as a Google drive link here. There are 4 columns -> N, x, |f(x), |f'(x)| (although computing f'(x) wasn't necessary for the exercise). There is also a much larger file with complex valued f(x) and f'(x) which can be shared if needed.
Some interesting rows in the file:
N, x, |f(x)|, |f'(x)|, remark
151, 286834.3928, 0.5097, 0.4773, min |f(x)|
152, 291266.7808, 2.9525, 3.7539, max |f(x)|
191, 462997.2589, 0.6210, 0.1150, min |f'(x)|
155, 304719.4104, 2.9371, 3.8945, max |f'(x)|
Average |f(x)| and |f'(x)| values were 0.9617 and 0.6104 respectively.

Terence Tao · 10 March 2018

Thanks for this! I assume $f(x)$ is $A^{eff} + B^{eff}/B_0^{eff}(x)$. So if I understand correctly, your computation verifies that this quantity is at least 0.065 for all x in the range $151 \leq N \leq 300$? Combined with the upper bounds for $E_1 + E_2 + E_3^*/B_0^{eff}$ from this table, this seems to finish off the verification of $H_t(x+iy) \neq 0$ in this range. We also have the range $N \geq 2000$ covered, and I believe your previous analysis of lower bounds for $A^{eff} + B^{eff}/B_0^{eff}$ should also handle the range $300 < N \leq 2000$ (do you have a table for those bounds by the way?), so this would just leave $N \leq 150$ as the remaining range. Presumably the mesh method should be able to push down a bit further?

KM · 11 March 2018

Yeah, I used f(x) = ((A_eff+B_eff)/B0_eff)(x,y=0.4,t=0.4) with y and t treated as constants. I have kept the table with N dependent lower bounds of |A_eff+B_eff|/|B0_eff| here.
Some of the mollifier based bounds for larger N are unfilled, since they were slow to compute. I will update the table with them later. At meaningful heights N, there is practically no difference between the toy and the effective bounds.
Rudolph was handling the mesh for N=20 to 150, and I think he will be commenting soon.

Rudolph · 11 March 2018

The mesh for N=20 to 150 is complete and I have transferred the detailed results to KM for inspection and post-processing.

KM · 11 March 2018

For N=20 to 150, the smaller file with 4 columns N,x,|f(x)|,|f'(x)| is kept here, and the larger raw file with structure Sr.no, N, ddx_upper_bound, c, x, f(x), |f(x), f'(x), |f'(x)| is here.
Interesting rows in the file:
N, x, |f(x)|, |f'(x)|, remark
20, 5484.3940, 0.3553, 0.5006, min |f(x)|
21, 5637.7493, 4.1903, 4.7830, max |f(x)|
27, 9179.2848, 0.5447, 0.0550, min |f'(x)|
33, 13850.9274, 4.1195, 5.0151, max |f'(x)|
Interestingly, we also checked whether the N=11 to 19 range could be tackled with a higher c value. We set c to 0.165 (since bound |E/B0| at N=11 is 0.1633). These were the results obtained.
N, x, |f(x)|, |f'(x)|, remark
13, 2166.6170, 0.3106, 0.4819, min |f(x)|
Hence this N range too may not need to go through direct evaluation.
The data for this range is kept here

Terence Tao · 11 March 2018

Thanks for all the data! I have put links to them at http://michaelnielsen.org/polymath1/index.php?title=Polymath15_test_problem . If the $A^{eff} + B^{eff}$ approximation works for N as low as 11, then we only need to evaluate H_t numerically up to $x = 4\pi(11)^2 - \pi t/4 \approx 1500$, which looks within reach.
I have started to write up in LaTeX form the explicit bounds and this should appear on the git repository soon. In the process of cleaning things up I think I can make a slight

improvement to the E_1, E_2 bounds and also obtain an error estimate for $H_t - (A^{eff} + B^{eff} - C^{eff})$, though the way things are going we may not even need this improved approximation. But I am thinking that our paper should have two purposes: firstly to record a number of effective estimates for H_t, and secondly to bound the de Bruijn-Newman constant. The former part will probably be useful for any future work on improving the constant. (We can also use these estimates to give some effective versions of the theorems in Ki-Kim-Lee, for instance giving an explicit threshold beyond which all the zeroes of H_t are provably real.)



Rudolph · 9 March 2018

Below are some graphs that indeed show that the choice of bounds for very large x leaves sufficient room to reduce N a bit below 2000 and/or play with the fixed bound itself. Only E_3^* looks a bit tight, although it is difficult to tell from just looking at a graph at these scales.
$\left|A^{eff} + B^{eff}\right|$ https://ibb.co/gZHBQ7
$E1, E2$ https://ibb.co/mR7vk7
$E3$ https://ibb.co/dZ7T57



Anonymous · 9 March 2018

What should be the optimal number P of factors in an Euler product type moliffier as a function of the number N of terms in the approximating sums?



Sylvain JULIEN · 10 March 2018

I have a probably very naive question due my huge lack of understanding of the problem. It seems $ \Lambda $ is defined from the zeros of some function $ H(\lambda,z) $ which are all real iff $ \lambda\geq\Lambda $. Call such a function depending on the two parameters $ \lambda $ and $ z $ whose all zeros are real iff $\lambda\geq\Lambda $ a $ \Lambda $ -function. Could the structure of the automorphism group of the set of all $ \Lambda $ -functions shed a light on the actual value of $ \Lambda $?



Anonymous · 12 March 2018

Dear Terry,
Be joyful and relax some minutes.Can you guess who will win Abel prize on March 20, 2018.I predict the mathematician whose name has 10 words will be able to get it.



Anonymous · 12 March 2018

The function $H_{t^2}(z)$ is symmetric also in t, and since it has the same information on its zero set, it seems (slightly) better to work with (than $H_t(z)$).



Rudolph · 13 March 2018

One of the secondary goals that was established for this polymath project, is to gain better understanding (both analytic and numerical) of H_t. In this context, I made two implicit plots; one in 3D for $\Re(H_t(x+yi)) = 0$ and a 2D-slice of it for $\Re(H_t(x+0i)) = 0$. The 3D plot just nicely illustrates the 'scary beauty' of the complexity we are dealing with.
The 2D plot shows, as predicted, that the zeros move a bit to the left when t increases above 0, however what I hadn't expected, is that for larger $t \geq 10$, the zeros seem to 'go full circle' (or 'collide'). Furthermore, there appears to be very regular, monotonically increasing pattern emerging for the zeros at the tops of these 'closed strings'. Also checked this for higher x and the pattern firmly remains. Could there be a logical explanation for the zeros 'vanishing' in such a regular pattern when t growth large?

https://ibb.co/mG15QH



Terence Tao · 13 March 2018

The behaviour at $t \geq 10$ is very odd, and not what I would expect asymptotically at all. What I would have expected was that the $A_0^{eff} + B_0^{eff}$ term of $A^{eff} + B^{eff}$ would dominate at this value of t, and the zeroes of this sum should arrange like an arithmetic progression marching steadily leftwards as t increases. Would it be possible to run the same computation using $A_0^{eff} + B_0^{eff}$ in place of $A^{eff} + B^{eff}$? (here by A_0^{eff} I mean the $n = 1$ term of A^{eff}). Presumably the same phenomenon also holds for the other approximations $A + B$, $A' + B'$, $A^{toy} + B^{toy}$ given as they all seem to be rather close to each other. Finally when $y = 0$ all of the approximations should be essentially real, the imaginary part should basically be only because of numerical error.



Rudolph · 14 March 2018

That explains it indeed! The odd behaviour disappears when using $A_0^{eff} + B_0^{eff}$ for t large and after comparison with some actual values of H_t (using the integral), I found that the strange pattern is caused by the error terms $E_{1,2,3}$ that start to dominate when $t \geq 10$ (in this x-range).

Attached is a new plot that compares $A^{eff} + B^{eff}$ including all error terms (red), with the error terms only switched on for $-1 \leq t \leq 1$ (green). When t increases, the zeros are now indeed 'marching steadily leftwards' wearing little green berets :)

https://ibb.co/ndhwZc



David Bernier (@doubledeckerpot) · 16 March 2018

Is there a reliable upper bound on the "typical" value of | log(| H_{0.4}(x+0.4*i)|)/log(10) | ?

This might help in setting the local precision in integral calculations involving Φ or the second integral method.

The code below, which adjusts calculation precision with "x", agrees with Anonymous' $H_{0.4}$ values at 100+0.4*i and 300+0.4*i:

(based on KM's code):

? for(K=5,30,localprec(10*K); t=0.4; y = 0.4; x = 10*K; print(precision(x,19)," ", precision(Ht_integral(x),19)))

50 -2.1047124718785808346 E-8 − 3.006030784162775084 E-8*I
60 -8.797417339471680766 E-10 + 1.0580539791535436879 E-9*I
70 -2.355154931010125629 E-10 + 2.8393742774037032383 E-11*I
80 2.2576611190131208935 E-12 − 8.927938709799100864 E-13*I
90 1.6901585151951934327 E-13 − 1.5317581587046806339 E-14*I
100 6.702152217279126684 E-16 + 3.133796584070556925 E-16*I
110 -7.364834657345354450 E-17 + 1.7548391366479612142 E-17*I
120 -5.258352161799536371 E-19 + 8.099278955197521748 E-20*I
130 -5.297614494427621701 E-21 − 5.872076288890267898 E-21*I
140 -4.552049395696165212 E-22 − 6.099763216616073607 E-23*I
150 7.057048184192585515 E-24 − 5.048132691648078404 E-24*I
160 -3.894071621449979314 E-25 − 3.790281812824059439 E-26*I
170 -3.570556479082531282 E-27 − 1.0730598815421024207 E-27*I
180 -2.959277374063902186 E-28 + 9.529944891026918017 E-30*I
190 -1.3400959121627429409 E-30 + 3.570717316287886905 E-32*I
200 -1.0360928396386564604 E-31 + 1.9808093691630687907 E-32*I
210 -4.747677152569013952 E-34 + 3.659779803543006900 E-34*I
220 -3.489228591177793899 E-35 + 1.8401441099778526225 E-35*I

230 6.649995257466365986 E-37 − 4.386090012622009025 E-38*I
240 2.1731405677097774617 E-38 − 4.979808010644137820 E-39*I
250 -4.595766631889382604 E-40 − 4.992715219111431987 E-41*I
260 -4.472080722580684381 E-42 + 1.5078116247080648595 E-43*I
270 8.201349077729028997 E-44 + 4.900772781466248864 E-44*I
280 1.4176271036085426800 E-45 + 1.5787181626920506794 E-46*I
290 9.548085707499014585 E-47 − 3.744714763671142931 E-47*I
300 -4.015967420625229432 E-49 − 1.4006524430296841232 E-49*I



David Bernier (@doubledeckerpot) · 16 March 2018

Judging by the values of $H_t(x+0.4i)$, I'd suggest a real precision close to floor(x/5)+20.
As I wrote on the mailing list, for x = 1000, one has to be careful about the integral step, e.g.:
? for(K=100,100,localprec(2*K); t=0.4; y = 0.4; x = 10*K; print(precision(x,9)," ",
precision(Ht_integral(x),9)))
1000 1.1947262584148883838 E-169 + 3.0568169693909478745 E-167*I // inaccurate
? MSTEP
%108 = 2
same with MSTEP=4:
? for(K=100,100,localprec(2*K); t=0.4; y = 0.4; x = 10*K; print(precision(x,9)," ",
precision(Ht_integral(x),9)))
1000 1.4847586909758082343 E-169 + 3.0506306236221504103 E-167*I //accurate
? MSTEP
%111 = 4
This is where MSTEP appears as a parameter influencing the integration step:
I_t_th =
(t,th,X,b,beta)->return(intnum(u=I*th,I*th+X,exp(t*u^2-beta*exp(4*u)+I*b*u),MSTEP))
?intnum for on-line help with intnum:
intnum(X=a,b,expr,{tab}): numerical integration of expr from a to b with
respect to X. Plus/minus infinity is coded as +oo/-oo. Finally tab is either
omitted (let the program choose the integration step), a non-negative integer
m (divide integration step by 2^m), or data precomputed with intnuminit.
===
Does n0 increase with "x" in KM's code?
Ht_integral =
(x,y=0.4,t=0.4,n0=26,X=6)->th=Pi/8-atan((9+y)/x);
[I seem to recall KM had n0=6, so I might try n0 =6, x=1000, MSTEP=4.]



Anonymous · 17 March 2018

It seems more natural to construct the adaptive grid points using the derivative (with respect to x) of $\log|H_t(x+iy)|$ (instead of $H_t(x+iy)$ derivative) which has much smaller dynamic variation with respect to x (for fixed $t, y > 0$).



Anonymous · 17 March 2018

In fact, to verify that $H_t(x+iy) \neq 0$ (for fixed $t, y > 0$), it is sufficient to prove the nonvanishing of $|H_t(x+iy)|$, or even better, the lower boundedness of $\log|H_t(x+iy)|$ which is a real function of x with slow dynamics (thereby more suitable for graphs or tables.)

Anonymous · 17 March 2018

What computations or bounds remain to be calculated or derived so that the test problem can be finished off? I noticed that one of the github issues is serving as a de facto mailing list and at least one of the participants is no longer able to post on this blog for technical(?) reasons https://github.com/km-git-acc/dbn_upper_bound/issues/50. This thread is approaching 100 comments and it's no longer clear to me which parts of the problem are being most actively worked on, or if some parts mentioned in the blog or on the github are now dead ends.

KM · 18 March 2018

Rudolph had recently given a good summary here
https://github.com/km-git-acc/dbn_upper_bound/issues/50#issuecomment-373954432
Essentially, we are right now working on clearing the x range upto 1600 (N~ 11). After optimizations on several aspects, we are planning to finish this mesh exercise in the next few days (barring maybe the very small x where the I integral diverges, but we are planning to replace it with the original integral for that range).

Anonymous · 18 March 2018

I'll plan to evaluate Ht with t=y=0.4 from x=1000 to x=1600 with spacing of 0.005 which should take maybe like a day using code like in https://terrytao.wordpress.com/2018/03/02/polymath15-fifth-thread-finishing-off-the-test-problem/#comment-493973.

KM · 18 March 2018

Wow, that's serious speed. Is it because of the library or the machines available to you? Will check out your script as well.
While a spacing of 0.005 would certainly work, the average gap is around 0.05 with an adaptive mesh, so you could consider using that approach to get even faster results.

Anonymous · 19 March 2018

This is finished now, but the file it made is too big for the internet to handle. It didn't fit on pastebin.com so I made a github account which was immediately flagged (hidden from public) for some reason before I even did anything on it. Then I tried uploading the file as a github gist with that flagged account, but the file is truncated in the plain gist and the associated link to the raw untruncated file is 404.

rudolph01 · 19 March 2018

Amazing job, Anonymous! You could try to use Google-drive. It offers you 15Gb for free and is easy to install. You can just upload a file directly from your PC and with a 'right click' you get create and publish a 'shareable link'.

Anonymous · 19 March 2018

I just received an email from github saying that they manually reviewed the account and cleared the flag, so maybe the file is visible now at https://gist.githubusercontent.com/p15-git-acc/b0548c398183233e654956b450f36c85/raw/36581e3dd6c9adb4e034e08e6551018555e3b361/gistfile1.txt

KM · 19 March 2018

Great. I will run the derivative bound check, and add an indicator column which points clear the check (there should be few if any points where it doesn't, which can be tackled easily)

KM · 22 March 2018

Using your script, David was able to generate H_t data for 10<=x<1000 with a fixed mesh of step size 0.01. To that I also added the data for x<10 with the same step size.
After checking the intervals between the mesh points using the derivative bound, I think we can clear this range.
The verified output has been kept here
The indicator column has three possible values 0,1 and 2. 1 means the allowed step was greater than the fixed step, so no additional work was required. 0 means the allowed step was less than the fixed step. There were 96 such points or about 0.1%. Near those points, additional mesh points were added which have an indicator value of 2.
for x<10, theta_c used was Pi/8-(1/4)*atan(9.4/30)
for x<100, theta_c used was Pi/8-(1/4)*atan(9.4/100)
and after that, theta_c was varied when x crossed a multiple of 2. These values could have been chosen differently.
ble.
The data for 1000 less than x less than 1600 should be on the way.
There was a lot of tinkering the last few days, with some learnings for the way ahead.
1. Arb (and possibly compiled PariGP) is much faster for both fixed and adaptive meshes than other tools.
2. Data from a fixed mesh requires to be then verified against the derivative bound to clear the intervals between mesh points. To improve verification speed, one can skip points and choose the next point just below the one recommended by an adaptive approach.
3) Also, one could change theta_c and D values periodically instead of changing them at every x (since computing D for many points takes time).
with my conclusion being the fastest approach would be an adaptive mesh with an Arb like library.

rudolph01 · 22 March 2018

Building on KM's comments.
The numerical verification process, to ensure that $H_{0.4}(x+0.4i)$ doesn't vanish in the domain $x = 1000...1600$, has now been completed using a fixed mesh approach with steps of 0.005. The mesh-runs went in two batches and the output is on the links below.
[batch x=1000..1300](https://drive.google.com/open?id=1UE6khYy0k4IGEL1Joi9ipltTCeSgOAUW)
min. mesh : 0.00743669151431375 at x= 1275.695
max. mesh : 0.10845848437334255, ‘ at x= 1052.37
#mesh below 0.01: 1355 times, out of: 60001
[batch x=1300..1600](https://drive.google.com/open?id=16E-TiGtC8JXnFoyxzOEzXIZaD8Tdxn8H)
min. mesh : 0.006968664138389287, ‘ at x=’, 1423.495)
max. mesh : 0.10180996783467539, ‘ at x=’, 1304.12)
#mesh below 0.01: 3134 times, out of: 60001
File csv-layout: x, H_t, interval_cleared?(1=y,0=n), H_t/B0_eff, H_t_ABC, theta_c, D, Ht_derivative_bound/B0_eff, newton_quotient (of H_t_ABC/B0_eff)
We also managed to stretch the adaptive mesh, that was used to clear $N = 11...300$, to now also include $N = 8...10$ $(x = 803...1520)$. Below is a plot outlining the c-parameter choices we made and also the links the batch output.
https://ibb.co/dMe6fH

[N=8](https://drive.google.com/open?id=1EfNA54bI8ghM4Y9kDVqfsMVW3_Q8yMaQ)
[N=9](https://drive.google.com/open?id=1H4BSflE6A1JMvIDzAal_Ru5xGsBK9NBw)
[N=10](https://drive.google.com/open?id=14V1p-Cn9a3zg69LFtptHuWs5mis8uGvH)
File csv-layout: seq_nr, N, Ht_AB_derivative_bound, c_used, x-mesh, Ht_AB, abs_Ht_AB, Derivative_Ht_AB, abs_Derivative_Ht_AB

 **Polymath15, sixth thread: the test problem and beyond | What's new** · 18 March 2018

[...] thread of the Polymath15 project to upper bound the de Bruijn-Newman constant , continuing this post. Discussion of the project of a non-research nature can continue for now in the existing proposal [...]

 **Anonymous** · 19 March 2018

I tried carefully reading the math in equations (4) in http://michaelnielsen.org/polymath1/index.php?title=Bounding_the_derivative_of_H_t_-_second_approach, and it looks like it uses elementary but clever tricks to find the sum from n=n0 to n=infinity of (n^4*c^n). Maybe this is what had been referred to in https://terrytao.wordpress.com/2018/03/02/polymath15-fifth-thread-finishing-off-the-test-problem/#comment-493623 as a generalized geometric series formula. Does anyone have wikipedia links or other references for these formulas? My google skills aren't good enough.

 **Terence Tao** · 19 March 2018

Differentiating the geometric series formula $\sum_{n=0}^\infty x^n = \frac{1}{1-x}$ term by term (assuming $|x| < 1$ throughout) yields

$$\sum_{n=0}^\infty (n+1)x^n = \sum_{n=1}^\infty n x^{n-1} = \frac{1}{(1-x)^2}$$
$$\sum_{n=0}^\infty (n+2)(n+1)x^n = \sum_{n=2}^\infty n(n-1)x^{n-2} = \frac{2}{(1-x)^3}$$
$$\sum_{n=0}^\infty (n+3)(n+2)(n+1)x^n = \sum_{n=3}^\infty n(n-1)(n-2)x^{n-3} = \frac{6}{(1-x)^4}$$
$$\sum_{n=0}^\infty (n+4)(n+3)(n+2)(n+1)x^n$$
$$= \sum_{n=4}^\infty n(n-1)(n-2)(n-3)x^{n-4}$$
$$= \frac{24}{(1-x)^5}$$

which after some linear algebra gives

$$\sum_{n=0}^\infty n x^n = \frac{x}{(1-x)^2}$$
$$\sum_{n=0}^\infty n^2 x^n = \frac{x^2+x}{(1-x)^3}$$
$$\sum_{n=0}^\infty n^3 x^n = \frac{x^3+4x^2+x}{(1-x)^4}$$
$$\sum_{n=0}^\infty n^4 x^n = \frac{x^4+11x^3+11x^2+x}{(1-x)^5}.$$

The general formula is

$$\sum_{n=0}^\infty n^k x^n = \frac{\sum_{m=0}^{k-1} A(k,m) x^{m+1}}{(1-x)^{k+1}}$$

for $k \geq 1$, where $A(k,m)$ are the Eulerian numbers.

 **Anonymous** · 20 March 2018

The number of twin primes Only 11-13, 41-13 this type of twin prime number
Natural number
The number of twin primes

ratio
00000000019 1
00000000029 1 1
00000000049 2 2
00000000089 3 1.5
00000000169 4 1.333333
00000000329 7 1.75
00000000649 11 1.571429
00000001289 17 1.545455
00000002569 27 1.588235
00000005129 46 1.703704
00000010249 68 1.478261
00000020489 119 1.75
00000040969 205 1.722689
00000081929 346 1.687805
00000163849 604 1.745665
00000327689 1082 1.791391
00000655369 1938 1.791128
00001310729 3442 1.776058
00002621449 6220 1.807089
00005242889 11265 1.811093
00010485769 20652 1.833289
00020971529 37550 1.818226
00041943049 68405 1.821704
00083886089 125868 1.840041
00167772169 232302 1.8456
00335544329 429878 1.850514
00671088649 797856 1.856006
01342177289 1486882 1.863597
02684354569 2776461 1.867304
05368709129 5195786 1.8714
10737418249 9747328 1.8760
How do you explain that twin prime numbers are doubling as natural numbers grow?